

Final Project: Asynchronous Counter
CENG 2112.01 Lab for Digital Circuits

Submitted by
David Dulaney
2344357

Computer Science
University of Houston-Clear Lake
Houston, Texas 77058
May 5, 2026

Contents

Abstract	2
Introduction	2
Requirements	2
Implementation	2
Conclusion	3

Abstract

An asynchronous counter is a sequential digital circuit where every flip-flop after the first involved are not clocked simultaneously, rather the first flip-flop is driven by a clock pulse and each flip-flop in the circuit is triggered by the output of the flip-flop before it.

Introduction

A D-flip-flop (part # 7474) can be used in an asynchronous, the amount of D-flip-flops needed is based on how many bits the circuit requires to make it work. One D-flip-flop is used for one bit, so if used in a digital clock with a seven-segment display then there will be seven D-flip-flops used. Since they can be used in an asynchronous, a circuit can be built to make a counter up to any number up to ten.

Requirements

The requirement for this lab is to simulate an asynchronous circuit to make a counter that displays my student id number (2344357) on Multisim.

Implementation

The first step in implementation was to build a circuit in Multisim to simulate an effective asynchronous counter to display my student id using D-flip-flops.

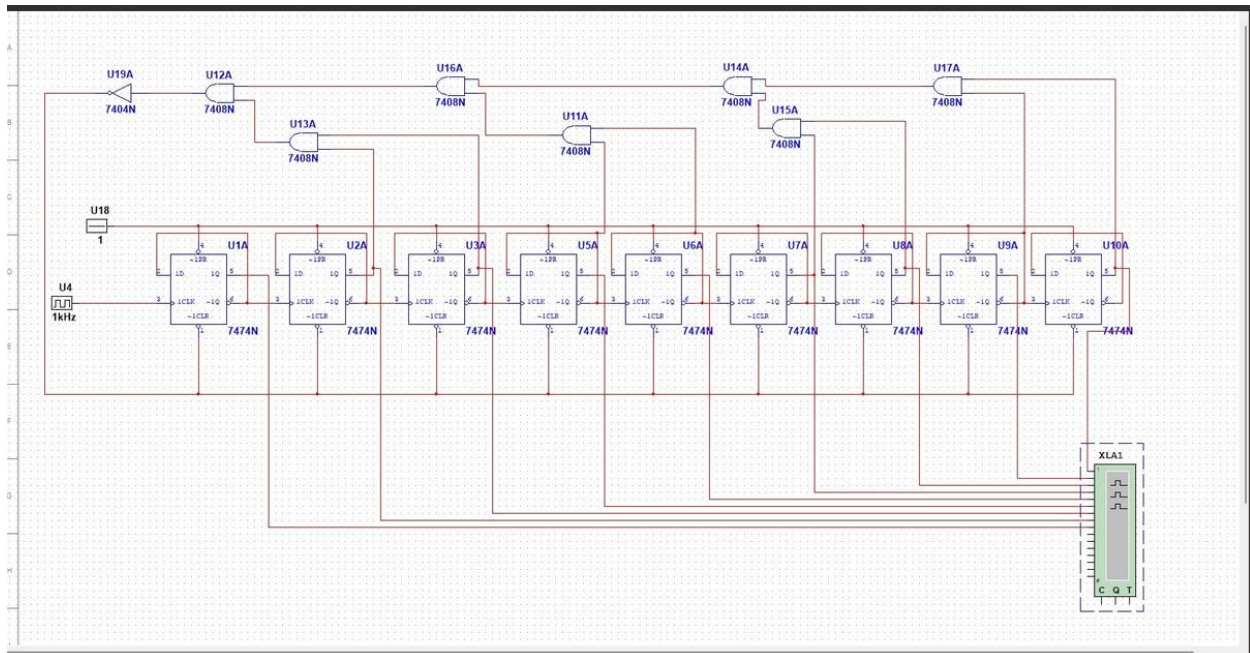


Figure 1.1: Asynchronous counter circuit to display my student id (2344357)

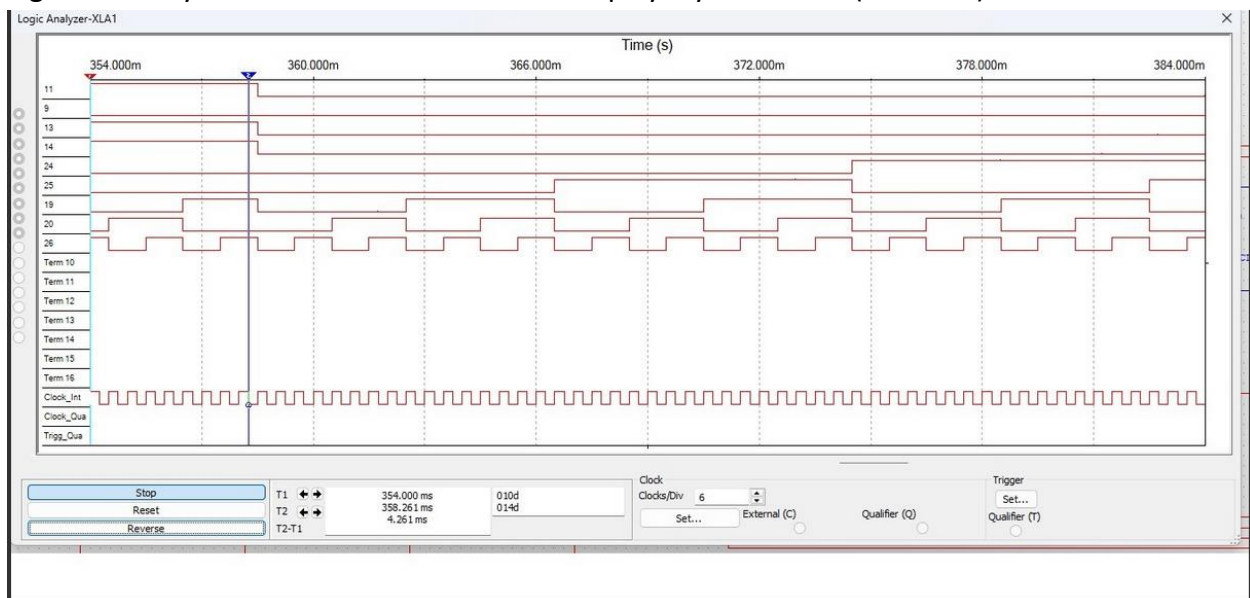


Figure 1.2: Logic analyzer of circuit, line shows where it starts showing my student id (2344357)

I chose to go with this circuit because combining the D flip-flops in groups of 2 using and gates and inverters based on the binary number of 357 (101100101) which is the last three digits of my id, effectively shows my full student id.

Conclusion

This project was designed to understand how an asynchronous circuit can work to display a series of numbers using D-flipflop. The main takeaway from this project is to understand that

you can group D flip-flops together to achieve a display of any number using binary bits. It also shows that each D-flip-flop needs a constant input into their 1PR slot and an inversion in their LCLR slot in order to assist when combining D flip-flops together to display the desired number